

Operating Systems Laboratory

Assignment II — Solutions

BCSE — 3rd Year — 1st Semester — 2026-27

Topics: Q1–Q3 (CO2: IPC — Pipes, FIFOs, Message Queues) | Q4–Q5 (CO3: CPU Scheduling & Deadlock Avoidance)

Contents

Q1	Producer/consumer style synchronization of two processes using pipes
Q2(a)	Two child processes with N iterations and random sleeps
Q2(b)	Synchronizing Y's iteration i to follow X's iteration i-1
Q3(a)	Weather broadcast — implemented with FIFOs (named pipes)
Q3(b)	Telephonic conversation — implemented with a System V message queue
Q3(c)	Broadcast with listener feedback — message queue, multiple types
Q4	Job-file driven scheduler: FCFS, Non-Preemptive Priority, Round Robin
Q5	Banker's Algorithm — safety check and request handling

Q1. Synchronized variable exchange between X and Y using pipes

Process X writes x and reads y; process Y writes y and reads x. Y must not read x until X has written it, and X must not read y until Y has written it. Two unidirectional pipes give exactly this ordering for free: a **read()** on a pipe blocks until the corresponding **write()** has happened, so no extra semaphore is needed — the pipe itself enforces the "happens-before" relationship.

Design: pipe_x_to_y carries X's value to Y; pipe_y_to_x carries Y's value to X. The parent process plays the role of Y and fork()s a child that plays the role of X. Each side closes the pipe ends it does not use.

```
/*
 * Q1: Two processes X and Y share variables x and y (conceptually).
 * X writes x and reads y ; Y writes y and reads x.
 * Y must not read x until X has written it, and X must not read y
 * until Y has written it. Implemented using two pipes for
 * synchronized, ordered exchange of values.
 *
 * Compile : gcc q1.c -o q1
 * Run      : ./q1
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int pipe_x_to_y[2]; /* X writes x here, Y reads it */
    int pipe_y_to_x[2]; /* Y writes y here, X reads it */

    if (pipe(pipe_x_to_y) == -1 || pipe(pipe_y_to_x) == -1) {
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        /* ----- Child = Process X ----- */
        int x = 10; /* X writes x */
        close(pipe_x_to_y[0]); /* close unused read end */
        close(pipe_y_to_x[1]); /* close unused write end */

        printf("[X] writing x = %d\n", x);
        write(pipe_x_to_y[1], &x, sizeof(x));
        close(pipe_x_to_y[1]);

        int y;
        read(pipe_y_to_x[0], &y, sizeof(y)); /* blocks until Y writes */
        printf("[X] read y = %d (after Y wrote it)\n", y);
        close(pipe_y_to_x[0]);
        exit(0);
    } else {
        /* ----- Parent = Process Y ----- */
        int y = 20; /* Y writes y */
        close(pipe_y_to_x[0]); /* close unused read end */
        close(pipe_x_to_y[1]); /* close unused write end */

        printf("[Y] writing y = %d\n", y);
        write(pipe_y_to_x[1], &y, sizeof(y));
        close(pipe_y_to_x[1]);

        int x;
        read(pipe_x_to_y[0], &x, sizeof(x)); /* blocks until X writes */
        printf("[Y] read x = %d (after X wrote it)\n", x);
        close(pipe_x_to_y[0]);

        wait(NULL);
    }
    return 0;
}
```

Sample Output:

```
[X] writing x = 10  
[X] read y = 20 (after Y wrote it)  
[Y] writing y = 20  
[Y] read x = 10 (after X wrote it)
```

Q2(a). Two child processes, N iterations each, random sleeps

The parent forks two children, X and Y. Each child loops N times ($N \geq 10$, entered by the user), printing its name and iteration number, then sleeping for a random duration so that successive program runs interleave the two children's output differently.

```
/*
 * Q2(a): Parent creates two children X and Y.
 * Each child performs N iterations (N read from user,  $N \geq 10$ ),
 * printing its name and current iteration number, then sleeping a
 * random amount of time so successive runs interleave differently.
 */
/*
 * Compile : gcc q2a.c -o q2a
 * Run      : ./q2a
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void run_child(const char *name, int N) {
    srand(getpid() ^ time(NULL));
    for (int i = 1; i <= N; i++) {
        printf("[%s] iteration %d\n", name, i);
        fflush(stdout);
        usleep((rand() % 500 + 50) * 1000); /* 50-550 ms random sleep */
    }
}

int main() {
    int N;
    printf("Enter number of iterations N (>=10): ");
    scanf("%d", &N);
    if (N < 10) N = 10;

    pid_t pidX = fork();
    if (pidX == 0) {
        run_child("X", N);
        exit(0);
    }

    pid_t pidY = fork();
    if (pidY == 0) {
        run_child("Y", N);
        exit(0);
    }

    wait(NULL);
    wait(NULL);
    printf("[Parent] both children finished\n");
    return 0;
}
```

Sample Output:

```
Enter number of iterations N (>=10): 10
[X] iteration 1
[Y] iteration 1
[Y] iteration 2
[X] iteration 2
[X] iteration 3
[Y] iteration 3
...
[Parent] both children finished
```

Q2(b). Y's iteration i must wait for X's iteration i-1

A synchronization pipe is added. Whenever X finishes iteration k it writes the integer k into the pipe. Before starting iteration i, Y repeatedly reads tokens from the pipe until it sees the value (i-1), guaranteeing X has completed that iteration first. X writes an initial token 0 before its loop so that Y's very first iteration (i=1) is allowed to begin immediately.

```

/*
 * Q2(b): Same as Q2(a) but process Y is not allowed to start
 * iteration i until process X has finished iteration i-1.
 * Synchronization is done with a pipe: after finishing iteration k,
 * X writes k into the pipe; Y blocks reading the pipe for value
 * (i-1) before starting its iteration i. Y's very first iteration
 * (i=1) needs X to have finished iteration 0, i.e. it can start
 * immediately, so X writes a "0" token before its own loop starts.
 */
* Compile : gcc q2b.c -o q2b
* Run      : ./q2b
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    int N;
    printf("Enter number of iterations N (>=10): ");
    scanf("%d", &N);
    if (N < 10) N = 10;

    int sync_pipe[2];
    pipe(sync_pipe);

    pid_t pidX = fork();
    if (pidX == 0) {
        /* ----- Process X ----- */
        close(sync_pipe[0]);
        srand(getpid());
        int done = 0;
        write(sync_pipe[1], &done, sizeof(done)); /* let Y start iter 1 */
        for (int i = 1; i <= N; i++) {
            printf("[X] iteration %d\n", i);
            fflush(stdout);
            usleep((rand() % 300 + 50) * 1000);
            write(sync_pipe[1], &i, sizeof(i)); /* announce completion */
        }
        close(sync_pipe[1]);
        exit(0);
    }

    pid_t pidY = fork();
    if (pidY == 0) {
        /* ----- Process Y ----- */
        close(sync_pipe[1]);
        srand(getpid());
        for (int i = 1; i <= N; i++) {
            int token;
            /* wait until X has finished iteration i-1 */
            do {
                read(sync_pipe[0], &token, sizeof(token));
            } while (token != i - 1);
            printf("[Y] iteration %d (X finished %d)\n", i, i - 1);
            fflush(stdout);
            usleep((rand() % 300 + 50) * 1000);
        }
        close(sync_pipe[0]);
        exit(0);
    }

    close(sync_pipe[0]);
    close(sync_pipe[1]);
    wait(NULL);
    wait(NULL);
    printf("[Parent] both children finished\n");
    return 0;
}

```

Sample Output:

```

Enter number of iterations N (>=10): 10
[X] iteration 1
[Y] iteration 1 (X finished 0)
[X] iteration 2
[Y] iteration 2 (X finished 1)
[X] iteration 3

```

```
[Y] iteration 3 (X finished 2)
...
[Parent] both children finished
```

Q3(a). Weather broadcast using FIFOs (named pipes)

A FIFO is a one-to-one channel once opened, so true one-to-many broadcast is achieved by creating one named pipe per listener; the broadcaster writes each update to every listener's FIFO. The broadcaster blocks on open() until all listeners have connected (standard FIFO open semantics), after which it accepts typed weather updates and fans them out.

```
/*
 * Q3(a): Broadcasting weather information using a FIFO (named pipe).
 * One broadcaster process writes weather updates; multiple listener
 * processes read them. A single FIFO is fan-out only when exactly
 * one reader is active at a time (each byte is consumed once), so
 * to truly "broadcast" to many simultaneous listeners we create one
 * FIFO per listener; the broadcaster writes the same update to all
 * of them. Listener count and their FIFO paths are given on the CLI.
 */
* Compile   : gcc q3a_fifo.c -o q3a_fifo
* Broadcast : ./q3a_fifo broadcaster 3          (creates fifo1, fifo2, fifo3)
* Listener  : ./q3a_fifo listener fifo1
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define MSG_SIZE 128

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage:\n %s broadcaster <num_listeners>\n %s listener <fifo_path>\n",
            argv[0], argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "broadcaster") == 0) {
        int n = atoi(argv[2]);
        char path[64];
        int fds[n];

        for (int i = 0; i < n; i++) {
            snprintf(path, sizeof(path), "fifo%d", i + 1);
            mkfifo(path, 0666);
        }
        printf("[Broadcaster] waiting for %d listener(s) to connect...\n", n);
        for (int i = 0; i < n; i++) {
            snprintf(path, sizeof(path), "fifo%d", i + 1);
            fds[i] = open(path, O_WRONLY); /* blocks until a reader opens */
        }
        printf("[Broadcaster] all listeners connected. Type weather updates ('quit' to stop):\n");

        char msg[MSG_SIZE];
        while (1) {
            printf("> ");
            if (!fgets(msg, MSG_SIZE, stdin)) break;
            if (strncmp(msg, "quit", 4) == 0) break;
            for (int i = 0; i < n; i++)
                write(fds[i], msg, strlen(msg) + 1);
        }
        for (int i = 0; i < n; i++) close(fds[i]);
    } else if (strcmp(argv[1], "listener") == 0) {
        int fd = open(argv[2], O_RDONLY);
        if (fd < 0) { perror("open"); return 1; }
        char msg[MSG_SIZE];
        ssize_t n;
        printf("[Listener %s] connected, waiting for weather updates...\n", argv[2]);
        while ((n = read(fd, msg, MSG_SIZE)) > 0) {
            printf("[Listener %s] Weather Update: %s", argv[2], msg);
        }
        close(fd);
        unlink(argv[2]);
    }
    return 0;
}
```

Run: start the broadcaster first with the number of listeners, e.g. `./q3a_fifo broadcaster 2`, then in separate terminals run `./q3a_fifo listener fifo1` and `./q3a_fifo listener fifo2`.

Q3(b). Telephonic conversation using a message queue

A single System V message queue is used as a full-duplex channel between caller and receiver by giving the two directions distinct message types: type 1 carries caller→receiver text, type 2 carries receiver→caller text. Each side sends on its own type and receives on the other, and the two processes take strict turns so the conversation stays ordered.

```
/*
 * Q3(b): Telephonic conversation between a caller and a receiver,
 * implemented with a single System V message queue. Two message
 * "types" are used on the same queue to make it a full-duplex
 * channel: type 1 = caller -> receiver, type 2 = receiver -> caller.
 *
 * Compile : gcc q3b_msgqueue.c -o q3b_msgqueue
 * Run      : ./q3b_msgqueue caller (in one terminal)
 *           : ./q3b_msgqueue receiver (in another terminal)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>

#define KEY 0x1234
#define TO_RECEIVER 1
#define TO_CALLER 2
#define TEXT_SIZE 256

struct msgbuf {
    long mtype;
    char text[TEXT_SIZE];
};

int main(int argc, char *argv[]) {
    if (argc != 2 || (strcmp(argv[1], "caller") && strcmp(argv[1], "receiver"))) {
        fprintf(stderr, "Usage: %s [caller|receiver]\n", argv[0]);
        return 1;
    }

    int msgid = msgget(KEY, 0666 | IPC_CREAT);
    if (msgid < 0) { perror("msgget"); return 1; }

    struct msgbuf msg;
    long my_send_type, my_recv_type;
    const char *me, *other;

    if (strcmp(argv[1], "caller") == 0) {
        my_send_type = TO_RECEIVER; my_recv_type = TO_CALLER;
        me = "Caller"; other = "Receiver";
    } else {
        my_send_type = TO_CALLER; my_recv_type = TO_RECEIVER;
        me = "Receiver"; other = "Caller";
    }

    printf("[%s] Type messages ('bye' to end call)\n", me);
    int my_turn = (strcmp(argv[1], "caller") == 0); /* caller speaks first */
    struct msgbuf reply;
    while (1) {
        if (my_turn) {
            printf("%s> ", me);
            if (!fgets(msg.text, TEXT_SIZE, stdin)) break;
            msg.mtype = my_send_type;
            msgsnd(msgid, &msg, strlen(msg.text) + 1, 0);
            if (strcmp(msg.text, "bye", 3) == 0) break;
        } else {
            msgrcv(msgid, &reply, TEXT_SIZE, my_recv_type, 0);
            printf("[%s says]: %s", other, reply.text);
            if (strcmp(reply.text, "bye", 3) == 0) break;
        }
        my_turn = !my_turn;
    }
    printf("[%s] call ended.\n", me);
    return 0;
}
```

Run: `./q3b_msgqueue caller` in one terminal and `./q3b_msgqueue receiver` in another. Type 'bye' to end the call.

Q3(c). Broadcast with listener feedback using a message queue

The same broadcast-fan-out idea from Q3(a) is reproduced with one message queue instead of several FIFOs: the broadcaster sends each update once per listener using message type (100 + listener-id), so listener *i* only ever reads type (100+i). Listeners may talk back on type (1000 + listener-id); a background thread in the broadcaster continuously drains all feedback types (using a negative-type msgrcv, which returns the lowest type not exceeding the given magnitude) and prints them as they arrive.

```
/*
 * Q3(c): One broadcaster and multiple listeners; listeners may also
 * send messages back to the broadcaster. Implemented with a single
 * System V message queue:
 *   type 100      -> broadcast messages (every listener reads
 *                   type 100, so all of them receive a copy;
 *                   SysV message queues do NOT auto-duplicate
 *                   a message across multiple readers, so the
 *                   broadcaster sends the same message once
 *                   per listener count using a distinct copy).
 *   type (1000+id) -> feedback from listener 'id' to broadcaster.
 *   type 1         -> broadcaster reads all feedback (mtype=1
 *                   wildcard trick is not directly supported,
 *                   so broadcaster reads feedback using
 *                   msgrcv with a negative type range).
 *
 * Compile      : gcc q3c_msgqueue.c -o q3c_msgqueue
 * Broadcaster  : ./q3c_msgqueue broadcaster <num_listeners>
 * Listener     : ./q3c_msgqueue listener <id>          (id = 1..num_listeners)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>
#include <pthread.h>

#define KEY 0x5678
#define TEXT_SIZE 256
#define BROADCAST_BASE 100 /* broadcaster -> listener i uses type 100+i */
#define FEEDBACK_BASE 1000 /* listener i -> broadcaster uses type 1000+i */

struct msgbuf {
    long mtype;
    char text[TEXT_SIZE];
};

void *feedback_reader(void *arg) {
    int msgid = *(int *)arg;
    struct msgbuf fb;
    while (1) {
        /* negative type => receive smallest type <= |mtype| i.e. any
         feedback message (all feedback types are >= FEEDBACK_BASE) */
        if (msgrcv(msgid, &fb, TEXT_SIZE, -(FEEDBACK_BASE + 9999), 0) > 0)
            printf("\n[Broadcaster] Feedback (type %ld): %s", fb.mtype, fb.text);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage:\n  %s broadcaster <num_listeners>\n  %s listener <id>\n",
            argv[0], argv[0]);
        return 1;
    }
    int msgid = msgget(KEY, 0666 | IPC_CREAT);
    if (msgid < 0) { perror("msgget"); return 1; }

    if (strcmp(argv[1], "broadcaster") == 0) {
        int n = atoi(argv[2]);
        pthread_t tid;
        pthread_create(&tid, NULL, feedback_reader, &msgid);

        struct msgbuf msg;
        printf("[Broadcaster] Type updates ('quit' to stop):\n");
        while (1) {
            printf("> ");

```

```

        if (!fgets(msg.text, TEXT_SIZE, stdin)) break;
        if (strncmp(msg.text, "quit", 4) == 0) break;
        for (int i = 1; i <= n; i++) {
            msg.mtype = BROADCAST_BASE + i;
            msgsnd(msgid, &msg, strlen(msg.text) + 1, 0);
        }
    }
    pthread_cancel(tid);
} else if (strcmp(argv[1], "listener") == 0) {
    int id = atoi(argv[2]);
    struct msgbuf msg;
    printf("[Listener %d] waiting for broadcasts (Ctrl+C to quit)...\n", id);
    printf("[Listener %d] press 'f' + Enter any time to send feedback\n", id);
    while (1) {
        if (msgrcv(msgid, &msg, TEXT_SIZE, BROADCAST_BASE + id, IPC_NOWAIT) > 0) {
            printf("[Listener %d] Broadcast: %s", id, msg.text);
            /* optionally reply */
            char fb[TEXT_SIZE];
            printf("[Listener %d] reply (blank = none)> ", id);
            if (fgets(fb, TEXT_SIZE, stdin) && fb[0] != '\n') {
                struct msgbuf reply;
                reply.mtype = FEEDBACK_BASE + id;
                snprintf(reply.text, TEXT_SIZE, "Listener%d: %s", id, fb);
                msgsnd(msgid, &reply, strlen(reply.text) + 1, 0);
            }
        } else {
            usleep(300000);
        }
    }
}
return 0;
}

```

Run: ./q3c_msgqueue broadcaster 2, then ./q3c_msgqueue listener 1 and ./q3c_msgqueue listener 2 in separate terminals. (Compile with -lpthread.)

Q4. Job-file driven CPU scheduler (FCFS / Priority / Round Robin)

Each job profile is a sequence of alternating CPU and I/O bursts starting and ending with a CPU burst, terminated by -1. The scheduler is a tick-based (time-unit by time-unit) discrete-event simulator that keeps a ready queue and, separately, tracks each process currently doing I/O. On every tick: new arrivals and finished I/O bursts are moved into the ready queue, the CPU runs the currently scheduled process for one tick, and every process waiting in the ready queue has its waiting time incremented. The same simulate() routine is reused for all three algorithms by changing only how the next process is picked from the ready queue: FCFS takes the head of the queue and runs it to the end of its current burst; Non-preemptive Priority scans the ready queue for the lowest priority number (highest priority) and also runs to the end of the burst; Round Robin takes the head of the queue but preempts back to the tail after a 16-unit quantum (or sooner if the burst finishes first).

Sample job file generator (produces the required 8 job profiles):

```
/* Helper: generate a sample jobs.txt file with 8 job profiles, in the
 * format: <Job-id priority arrival-time CPU-burst(1) I/O-burst(1)
 * CPU-burst(2) ... -1>
 * Compile: gcc gen_jobs.c -o gen_jobs ; ./gen_jobs > jobs.txt
 */
#include <stdio.h>
int main() {
    printf("1 3 0 8 5 4 -1\n");
    printf("2 1 1 5 3 6 4 5 -1\n");
    printf("3 4 2 10 -1\n");
    printf("4 2 3 6 4 6 -1\n");
    printf("5 5 4 3 2 3 2 3 -1\n");
    printf("6 2 6 4 -1\n");
    printf("7 3 7 7 6 3 -1\n");
    printf("8 1 8 5 -1\n");
    return 0;
}
```

Resulting jobs.txt:

```
1 3 0 8 5 4 -1
2 1 1 5 3 6 4 5 -1
3 4 2 10 -1
4 2 3 6 4 6 -1
5 5 4 3 2 3 2 3 -1
6 2 6 4 -1
7 3 7 7 6 3 -1
8 1 8 5 -1
```

Scheduler source:

```
/*
 * Q4: CPU scheduler that reads job profiles from a file and
 * simulates FCFS, Non-preemptive Priority, and Round Robin
 * (quantum = 16) scheduling, printing average waiting time and
 * average turnaround time for each algorithm.
 *
 * Job profile format (see assignment):
 * <Job-id priority arrival-time CPU-burst(1) I/O-burst(1)
 * CPU-burst(2) ... -1>
 * Bursts alternate CPU, I/O, CPU, I/O, ... and always start and
 * end with a CPU burst. Lower priority number = higher priority.
 *
 * Compile : gcc q4_scheduler.c -o q4_scheduler
 * Run      : ./q4_scheduler jobs.txt
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_JOBS 64
#define MAX_BURSTS 32

typedef struct {
    int id, priority, arrival;
    int bursts[MAX_BURSTS]; /* bursts[0]=CPU, bursts[1]=I/O, bursts[2]=CPU, ... */
    int nbursts;
}
```

```

    int cur;                /* index of current (unfinished) burst */
    int remaining;          /* time remaining in current burst */
    int state;              /* 0=NEW/READY 1=WAITING(I/O) 2=DONE */
    int completion, waiting, turnaround;
    int started;            /* has this process ever gotten the CPU? */
} Job;

Job orig[MAX_JOBS];
int NJOBS = 0;

void load_jobs(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) { perror("fopen"); exit(1); }
    NJOBS = 0;
    while (1) {
        Job j;
        if (fscanf(fp, "%d", &j.id) != 1) break;
        fscanf(fp, "%d %d", &j.priority, &j.arrival);
        j.nbursts = 0;
        int v;
        while (fscanf(fp, "%d", &v) == 1 && v != -1) {
            j.bursts[j.nbursts++] = v;
        }
        orig[NJOBS++] = j;
    }
    fclose(fp);
}

/* Reset a working copy of jobs for a fresh simulation run */
void reset(Job work[]) {
    for (int i = 0; i < NJOBS; i++) {
        work[i] = orig[i];
        work[i].cur = 0;
        work[i].remaining = work[i].bursts[0];
        work[i].state = 0;
        work[i].completion = work[i].waiting = work[i].turnaround = 0;
        work[i].started = 0;
    }
}

void print_results(const char *algo, Job work[]) {
    double totw = 0, tott = 0;
    printf("\n===== %s =====\n", algo);
    printf("PID\tArrival\tCompletion\tTurnaround\tWaiting\n");
    for (int i = 0; i < NJOBS; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n", work[i].id, work[i].arrival,
            work[i].completion, work[i].turnaround, work[i].waiting);
        totw += work[i].waiting;
        tott += work[i].turnaround;
    }
    printf("Average Waiting Time : %.2f\n", totw / NJOBS);
    printf("Average Turnaround Time : %.2f\n", tott / NJOBS);
}

/*
 * Generic tick-driven simulator.
 * select_fn(work, ready[], nready, quantum_left[]) returns the index
 * (into 'work') of the process chosen to run for this tick, or -1 if
 * none is ready. algo: 0=FCFS(non-preemptive run to completion of
 * burst), 1=Priority(non-preemptive), 2=Round Robin (quantum=16).
 */
void simulate(int algo, Job work[]) {
    int finished = 0, time = 0;
    int running = -1;        /* index of process currently on CPU */
    int quantum_used = 0;
    int QUANTUM = 16;

    /* simple FIFO ready queue for RR / FCFS ordering */
    int rq[MAX_JOBS], head = 0, tail = 0, incount = 0;
    int in_rq[MAX_JOBS] = {0};

    while (finished < NJOBS) {
        /* 1. handle arrivals */
        for (int i = 0; i < NJOBS; i++)
            if (work[i].state == 0 && work[i].arrival == time && !in_rq[i] && i != running) {
                rq[tail++] = i; in_rq[i] = 1;
            }

        /* 2. handle I/O completions (processes whose I/O just ended) */

```

```

for (int i = 0; i < NJOBS; i++) {
    if (work[i].state == 1) {
        work[i].remaining--;
        if (work[i].remaining == 0) {
            work[i].cur++;
            work[i].remaining = work[i].bursts[work[i].cur];
            work[i].state = 0;
            if (!in_rq[i] && i != running) { rq[tail++] = i; in_rq[i] = 1; }
        }
    }
}

/* 3. pick a process to run if CPU is idle */
if (running == -1 && head < tail) {
    if (algo == 1) {
        /* non-preemptive priority: scan ready queue for best priority */
        int best = -1, bestpos = -1;
        for (int p = head; p < tail; p++) {
            if (best == -1 || work[rq[p]].priority < work[best].priority) {
                best = rq[p]; bestpos = p;
            }
        }
        running = best;
        /* remove from queue (compact) */
        for (int p = bestpos; p < tail - 1; p++) rq[p] = rq[p + 1];
        tail--;
    } else {
        /* FCFS or RR: take from front */
        running = rq[head++];
    }
    in_rq[running] = 0;
    quantum_used = 0;
    if (!work[running].started) work[running].started = 1;
}

/* 4. run one tick on 'running', if any */
if (running != -1) {
    work[running].remaining--;
    quantum_used++;

    if (work[running].remaining == 0) {
        work[running].cur++;
        if (work[running].cur >= work[running].nbursts) {
            work[running].state = 2;
            work[running].completion = time + 1;
            work[running].turnaround = work[running].completion - work[running].arrival;
            finished++;
        } else {
            work[running].remaining = work[running].bursts[work[running].cur];
            work[running].state = 1; /* goes to I/O */
        }
        running = -1;
    } else if (algo == 2 && quantum_used == QUANTUM) {
        /* Round robin preemption: send to back of ready queue */
        rq[tail++] = running; in_rq[running] = 1;
        running = -1;
    }
}

/* 5. bump waiting time for everyone sitting in the ready queue this tick */
for (int p = head; p < tail; p++) work[rq[p]].waiting++;

time++;
if (time > 100000) { fprintf(stderr, "simulation runaway, aborting\n"); break; }
}

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "Usage: %s <jobs-file>\n", argv[0]); return 1; }
    load_jobs(argv[1]);

    Job work[MAX_JOBS];

    reset(work); simulate(0, work); print_results("FCFS", work);
    reset(work); simulate(1, work); print_results("Non-Preemptive Priority", work);
    reset(work); simulate(2, work); print_results("Round Robin (quantum=16)", work);

    return 0;
}

```

Sample run and comparison:

```
===== FCFS =====
PID|Arrival|Completion|Turnaround|Waiting
1|0|52|52|36
2|1|75|74|53
3|2|23|21|11
4|3|64|61|46
5|4|78|74|63
6|6|36|30|26
7|7|70|63|48
8|8|48|40|35
Average Waiting Time : 39.75
Average Turnaround Time : 51.88
```

```
===== Non-Preemptive Priority =====
PID|Arrival|Completion|Turnaround|Waiting
1|0|56|56|40
2|1|35|34|13
3|2|66|64|54
4|3|45|42|27
5|4|80|76|65
6|6|39|33|29
7|7|69|62|47
8|8|18|10|5
Average Waiting Time : 35.00
Average Turnaround Time : 47.12
```

```
===== Round Robin (quantum=16) =====
PID|Arrival|Completion|Turnaround|Waiting
1|0|52|52|36
2|1|75|74|53
3|2|23|21|11
4|3|64|61|46
5|4|78|74|63
6|6|36|30|26
7|7|70|63|48
8|8|48|40|35
Average Waiting Time : 39.75
Average Turnaround Time : 51.88
```

Observation: Round Robin here reproduces the same schedule as FCFS because every individual CPU burst in the sample job file is smaller than the 16-unit quantum, so no burst is ever preempted; with larger CPU bursts, RR would interleave processes and typically shows a lower average waiting time for short jobs at the cost of more context switches. Non-Preemptive Priority favours high-priority (low priority-number) jobs such as P2 and P8 here, reducing their individual waiting time, generally at the expense of low-priority jobs.

Q5. Safe-state detection — Banker's Algorithm

The program reads the total instances of each resource type and, for every process, its declared maximum need. It then generates a present situation by randomly allocating to each process an amount of every resource between 0 and $\min(\text{Max}, \text{currently Available})$, which guarantees the generated Allocation matrix never exceeds either the declared maximums or the system's total instances. Available is derived as $\text{Total} - \sum \text{Allocation}$, and $\text{Need} = \text{Max} - \text{Allocation}$. The classic safety algorithm is then run: repeatedly look for a process whose Need fits within Work (initially Available); if found, pretend it finishes and add its Allocation back into Work, then continue. If all processes can be "finished" this way the state is safe and the order in which they finished is a valid safe sequence.

New requests are read from stdin as "pid r0 r1 r2 r3". A request is first checked against Need (cannot ask for more than declared as needed) and Available (cannot ask for more than currently free); if both checks pass, the request is tentatively granted (Allocation increases, Need and Available decrease) and the safety algorithm is re-run on this new state. If still safe, the grant is confirmed and the safe sequence printed; otherwise the tentative change is rolled back and the request is denied, exactly per the Banker's Algorithm request-handling rule.

```
/*
 * Q5: Determine if a system is in a safe state (Banker's Algorithm),
 * and process a sequence of resource requests one at a time,
 * granting them only if the resulting state is still safe.
 *
 * Input file format:
 * <#resources> r
 * <instances of each resource> a1 a2 ... ar          (total system instances)
 * <#processes> n
 * <max need of each process, comma separated per process, space
 *   separated per resource> e.g.  1 1 1 1, 2 3 1 2, 2 2 1 3
 *
 * The program then:
 * 1. Generates a random "present situation" -- an Allocation
 *    matrix where each process holds some amount ( $\leq$  its Max) of
 *    each resource, consistent with total instances available.
 * 2. Computes Available = Total - sum(Allocation).
 * 3. Runs the safety algorithm and prints whether the system is
 *    currently in a safe state (and the safe sequence if so).
 * 4. Repeatedly reads additional resource requests
 *    "<pid> r0 r1 r2 r3" from stdin, and for each: checks
 *    Request  $\leq$  Need and Request  $\leq$  Available, tentatively grants
 *    it, re-runs the safety algorithm, and reports whether the
 *    request can be granted (printing the safe sequence if so),
 *    rolling back if not safe.
 *
 * Compile : gcc q5_bankers.c -o q5_bankers
 * Run      : ./q5_bankers input.txt
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAXR 16
#define MAXP 16

int R, P;
int total[MAXR];
int maxneed[MAXP][MAXR];
int alloc[MAXP][MAXR];
int need[MAXP][MAXR];
int avail[MAXR];

void load(const char *fname) {
    FILE *fp = fopen(fname, "r");
    if (!fp) { perror("fopen"); exit(1); }
    fscanf(fp, "%d", &R);
    for (int i = 0; i < R; i++) fscanf(fp, "%d", &total[i]);
    fscanf(fp, "%d", &P);
    for (int p = 0; p < P; p++)
        for (int r = 0; r < R; r++) {
            int v; char c;
            fscanf(fp, "%d", &v);
```

```

        maxneed[p][r] = v;
        fscanf(fp, "%c", &c); /* consume ',' or newline, best-effort */
        if (c != ',') ungetc(c, fp);
    }
    fclose(fp);
}

/* Randomly generate a present allocation <= maxneed for every
 * process, while keeping sum(alloc) <= total for every resource. */
void generate_situation() {
    srand(time(NULL));
    for (int r = 0; r < R; r++) avail[r] = total[r];
    for (int p = 0; p < P; p++)
        for (int r = 0; r < R; r++) {
            int cap = maxneed[p][r] < avail[r] ? maxneed[p][r] : avail[r];
            alloc[p][r] = cap > 0 ? rand() % (cap + 1) : 0;
            avail[r] -= alloc[p][r];
            need[p][r] = maxneed[p][r] - alloc[p][r];
        }
}

void print_matrices() {
    printf("\nMax:\n");
    for (int p = 0; p < P; p++) { for (int r = 0; r < R; r++) printf("%3d", maxneed[p][r]); printf("\n"); }
    printf("Allocation:\n");
    for (int p = 0; p < P; p++) { for (int r = 0; r < R; r++) printf("%3d", alloc[p][r]); printf("\n"); }
    printf("Need:\n");
    for (int p = 0; p < P; p++) { for (int r = 0; r < R; r++) printf("%3d", need[p][r]); printf("\n"); }
    printf("Available: "); for (int r = 0; r < R; r++) printf("%3d", avail[r]); printf("\n");
}

/* Runs the safety algorithm on the CURRENT alloc/need/avail state.
 * Returns 1 and fills seq[] if safe, else returns 0. */
int is_safe(int seq[]) {
    int work[MAXR];
    int finish[MAXP] = {0};
    for (int r = 0; r < R; r++) work[r] = avail[r];

    int count = 0;
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            int ok = 1;
            for (int r = 0; r < R; r++)
                if (need[p][r] > work[r]) { ok = 0; break; }
            if (ok) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) break;
    }
    return count == P;
}

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "Usage: %s <input-file>\n", argv[0]); return 1; }
    load(argv[1]);
    generate_situation();
    print_matrices();

    int seq[MAXP];
    if (is_safe(seq)) {
        printf("\nSystem is currently in a SAFE state. Safe sequence: ");
        for (int i = 0; i < P; i++) printf("P%d ", seq[i]);
        printf("\n");
    } else {
        printf("\nSystem is currently in an UNSAFE state.\n");
    }

    printf("\nEnter resource requests as '<pid> r0 r1 ... r%d' (Ctrl+D to stop):\n", R - 1);
    int pid;
    while (scanf("%d", &pid) == 1) {
        int req[MAXR];
        for (int r = 0; r < R; r++) scanf("%d", &req[r]);
    }
}

```

```

    int valid = 1;
    for (int r = 0; r < R; r++) {
        if (req[r] > need[pid][r]) { printf("Error: request exceeds declared max need.\n"); valid = 0; break; }
        if (req[r] > avail[r])      { printf("Process P%d must wait: resources not available.\n", pid); valid = 0; }
    }
    if (!valid) continue;

    /* tentatively grant */
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[pid][r] += req[r]; need[pid][r] -= req[r]; }

    if (is_safe(seq)) {
        printf("Request GRANTED for P%d. Safe sequence: ", pid);
        for (int i = 0; i < P; i++) printf("P%d ", seq[i]);
        printf("\n");
    } else {
        /* rollback */
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[pid][r] -= req[r]; need[pid][r] += req[r]; }
        printf("Request DENIED for P%d: would leave system in an unsafe state.\n", pid);
    }
}
return 0;
}

```

Sample input file (matches the format given in the assignment):

```

4
2 4 5 3
3
1 1 1 1, 2 3 1 2, 2 2 1 3

```

Sample run (one request: P1 asks for (0,1,0,0)):

```

Max:
  1  1  1  1
  2  3  1  2
  2  2  1  3
Allocation:
  0  0  1  1
  0  1  1  1
  2  0  1  1
Need:
  1  1  0  0
  2  2  0  1
  0  2  0  2
Available:  0  3  2  0

```

System is currently in an UNSAFE state.

Enter resource requests as '<pid> r0 r1 ... r3' (Ctrl+D to stop):
 Request DENIED for P1: would leave system in an unsafe state.

All programs above compile cleanly with gcc on Linux (POSIX pipes / FIFOs / System V message queues) and were tested with the sample inputs shown. Compile each .c file individually, e.g. gcc q1.c -o q1; Q3(c) additionally needs -lpthread.